Vulnerability Reference

Definitions, safe verification guidance, conceptual impact, and remediation notes for every vulnerability switch currently defined by the ShopEase PHP lab.

> **Safety and accuracy.** Use only an isolated local copy with synthetic accounts and data. This guide deliberately omits attack payloads, exploit recipes, and instructions for accessing other users' data or systems. “Impact” describes what a weakness could permit; it is not an instruction to cause it. A configured switch is not proof the application contains that weakness.

69configured switches

63enabled in config.php

6disabled in config.php

Reviewed against the PHP source and `VULNERABILITIES.md` on 23 October 2026. This is not a penetration test or certification of deployment behavior.

How to use this reference

The status beside each toggle describes the source review of the shop code, not availability of a training activity. Every configured toggle now has a contained simulation at `/safe_labs.php`, including switches set to false. The simulations use fixed examples and in-memory policy models; they do not activate a vulnerability in the shop.

After a simulation passes, a learner can claim a uniquely numbered certificate for that simulation by entering their name and email. This certifies completion of the contained simulation only; it is not a challenge flag, proof of exploitation, or evidence that the underlying vulnerability exists. The existing flag-based award flow remains separate.

For safe verification, run the named simulation or review its handler, use a throwaway local database, and confirm expected outcomes using benign synthetic records or a unit-level test. Do not test the public Render service, real users, third-party hosts, external files, or production data. The simulation does not execute SQL, shell commands, unsafe deserialization, XML parsing, filesystem reads/writes, outbound requests, scripts, or real authorization bypasses.

Exploit / impact is a high-level description of potential consequence, not a procedure. **Remediation** gives defensive engineering guidance. A completed simulation demonstrates a concept only; it is not evidence that the shop was exploited or that the underlying toggle is wired to a vulnerable sink.

Authentication, sessions, and SQL

Current source confirms SQL concatenation in search, login, and user-agent logging; plaintext demo passwords; session fixation; and unsafe remember-cookie deserialization. Other switches in this group are configured labels without a verified matching challenge.

VULN_SQLI_PRODUCT_SEARCH — enabled

Implemented

Definition: Untrusted search text is concatenated into a database query.

Safe test: In a local clone, inspect `api/search.php`; add a regression test that verifies user input is passed as a bound parameter and search results remain limited to matching synthetic products.

Impact: Query logic or records may be exposed or altered, depending on database permissions.

Remediate: Use prepared statements, least-privilege database credentials, and input-length limits.

VULN_IDOR_ORDER_VIEW — enabled

Implemented

Definition: An order is selected by identifier without consistently checking its owner.

Safe test: Seed two synthetic accounts and orders; unit-test that account A cannot retrieve account B's order through the ordinary learner route.

Impact: Another customer's order details and shipping information may be disclosed.

Remediate: Enforce ownership in the database query and return a non-revealing not-found response.

VULN_PATH_TRAVERSAL_INCLUDE — enabled

Not controlled by toggle

Definition: A path selected by a request may influence a server-side file include.

Safe test: Review `admin.php` in a local copy for a strict route allowlist. The include behavior is not governed by this switch; do not attempt to read arbitrary files.

Impact: Depending on path resolution and server configuration, unintended local code or files could be included.

Remediate: Map fixed route keys to fixed files; reject unknown keys before include.

VULN_COMMAND_INJECTION — disabled

Disabled / not implemented

Definition: Untrusted input is interpreted as part of an operating-system command.

Safe test: Confirm the switch is false and locate no command-execution sink in the PHP app; do not submit shell syntax.

Impact: If present, commands could run with the web process's privileges.

Remediate: Avoid shell execution; use a library/API with typed arguments and strict allowlists.

VULN_XXE_IMPORT — disabled

Disabled / incomplete module

Definition: An XML parser resolves attacker-controlled external entities.

Safe test: Verify the switch is false; treat the XML import module as incomplete and do not feed it external-entity documents.

Impact: A vulnerable parser may disclose local resources or make outbound requests.

Remediate: Disable DTDs and external entities, prohibit network access, and validate feed schemas.

VULN_PARAMETER_POLLUTION — disabled

Disabled / incomplete module

Definition: Duplicate or array-valued parameters are handled inconsistently across layers.

Safe test: Confirm false in config and inspect the incomplete discount handler; no working exercise is established.

Impact: Validation and business logic may act on different parameter values.

Remediate: Require one scalar value, reject duplicates/arrays, and canonicalize input once.

VULN_COUPON_STACKING — disabled

Disabled / incomplete module

Definition: Discount rules can be applied repeatedly when policy allows only one.

Safe test: Confirm false in config and that checkout has no complete coupon workflow.

Impact: Improper discounts could reduce order totals.

Remediate: Enforce eligibility and stacking policy server-side in a transaction.

VULN_REDOS — disabled

Disabled / incomplete module

Definition: A pathological regular expression can consume excessive CPU on crafted input.

Safe test: Confirm false and inspect the unfinished validator; do not use long adversarial strings.

Impact: A request may tie up worker capacity.

Remediate: Use linear-time patterns, bound input length, and set processing limits.

VULN_BROKEN_ACCESS_CONTROL — enabled

Toggle unused

Definition: A protected operation fails to enforce the required role or ownership check.

Safe test: Review admin and learner routes; the observed URL-quirk behavior is tied to `VULN_URL_QUIRKS`, not this switch.

Impact: A user may reach actions or records beyond their authorization.

Remediate: Centralize server-side authorization and test both allowed and denied roles.

VULN_HEADER_SQLI — enabled

Implemented

Definition: A request header is concatenated into a SQL statement in the logging helper.

Safe test: In local tests, assert the user-agent value is bound as data; use only a harmless synthetic header and inspect the test database.

Impact: Logging queries may fail or be altered; database exposure depends on privileges.

Remediate: Use a prepared insert and treat every header as untrusted input.

VULN_SQLI_LOGIN — enabled

Implemented + detected

Definition: Login credentials are concatenated into SQL; a successful bypass is compared with a normal credential check.

Safe test: In an isolated test database, use a synthetic account and assert that only the expected login succeeds and that a detected bypass opens the claim flow.

Impact: Authentication may be bypassed; the app can issue an award after detected bypass, but this detection does not cover other toggles.

Remediate: Use prepared statements and password hashing; keep authentication and award tests separate.

VULN_WEAK_PASSWORD_HASHING — enabled

Implemented (plaintext)

Definition: Demo passwords are stored and compared as plaintext rather than as password hashes.

Safe test: Inspect the disposable database schema/seed and assert credentials are stored with `password_hash()` output in a hardened test.

Impact: A database disclosure reveals reusable credentials.

Remediate: Hash with `password_hash()`, verify with `password_verify()`, and rotate exposed credentials.

VULN_NO_RATE_LIMITING — enabled

Not implemented as a toggle challenge

Definition: Authentication attempts are not throttled by account or source.

Safe test: Review login middleware and tests for bounded retry behavior; this switch has no verified implementation.

Impact: Automated guessing can increase account compromise risk.

Remediate: Add progressive throttling, monitoring, and recovery controls without enabling account lockout abuse.

VULN_PREDICTABLE_SESSIONS — enabled

Not implemented

Definition: Session identifiers are generated with insufficient unpredictability.

Safe test: Verify the runtime uses PHP's secure session ID generator and secure cookie settings; no predictable-

ID challenge is present.

Impact: Predictable IDs could permit session impersonation.

Remediate: Use cryptographically secure session IDs and secure, HttpOnly, SameSite cookies.

VULN_SESSION_FIXATION — enabled

Implemented

Definition: Login does not regenerate the session identifier when this toggle is enabled.

Safe test: In a local browser test, compare session IDs before and after authenticating a synthetic account; hardened behavior must rotate it.

Impact: An attacker who can establish a victim's session may retain access after login.

Remediate: Regenerate the session ID on authentication and privilege changes.

VULN_USERNAME_ENUMERATION — enabled

Not demonstrated

Definition: Login or recovery responses reveal whether an account exists.

Safe test: Compare response text and timing for synthetic known and unknown names; current ordinary login failures use the same message.

Impact: Account discovery may support targeted attacks.

Remediate: Use uniform responses and similar processing paths; rate-limit attempts.

VULN_REFLECTED_XSS — enabled

Not implemented in current pages

Definition: Request data is returned into HTML without context-aware encoding.

Safe test: Inspect output encoding and add a non-executable markup-escaping regression test; no reflected-XSS exercise is established.

Impact: Script execution in another user's browser could expose session-bound actions or page data.

Remediate: Contextually escape output and deploy a restrictive Content Security Policy.

VULN_STORED_XSS — enabled

Not implemented in current pages

Definition: Persisted user content is rendered as executable markup.

Safe test: Confirm no product review/comment submission-render path exists; add escaping tests if such features are introduced.

Impact: Stored content could execute in every viewer's browser.

Remediate: Encode at output, sanitize rich text with an allowlist, and use CSP.

VULN_CSRF — enabled

Unprotected actions exist; toggle unused

Definition: A browser may send an authenticated state-changing request without user intent.

Safe test: Review state-changing forms and requests for CSRF tokens; no cross-site requests are needed to confirm missing defenses.

Impact: A victim's browser could be induced to change state.

Remediate: Add per-session anti-CSRF tokens, SameSite cookies, and origin checks for sensitive actions.

VULN_OPEN_REDIRECT — enabled

Not implemented

Definition: A redirect destination is accepted from an untrusted request without validation.

Safe test: Search routes for request-derived `Location` headers; no working redirect challenge was found.

Impact: Users may be sent to an attacker-controlled destination under a trusted link.

Remediate: Use fixed internal route identifiers or strict same-origin destination validation.

Authorization, business logic, and workflow

Order IDOR and profile-edit IDOR have relevant code paths. URL quirks and the legacy quantity endpoint also have behavior. Several related names are not connected to those code paths.

VULN_IDOR_PROFILE_EDIT — enabled

Implemented; route-dependent

Definition: Profile editing accepts a user identifier without consistently requiring ownership or an admin role.

Safe test: In a disposable database, test authorized and unauthorized synthetic user IDs through the application's normal route; verify only the intended row changes.

Impact: Another account's editable properties may be changed.

Remediate: Authorize the user ID server-side on every read and update; allowlist editable fields.

VULN_PRICE_QUANTITY_TAMPERING — enabled

Not demonstrated in checkout

Definition: Client-supplied prices or quantities are trusted during purchase calculation.

Safe test: Inspect checkout and assert totals are recomputed from database prices and validated quantities; current checkout does recalculate prices.

Impact: If trusted, a buyer could underpay or create unintended quantities.

Remediate: Derive price server-side, validate stock and quantity, and calculate money with decimal-safe arithmetic.

VULN_MASS_ASSIGNMENT_CHECKOUT — enabled

Not demonstrated

Definition: Request fields are copied into a model without restricting sensitive properties.

Safe test: Inspect checkout's accepted field list; verify role, owner, status, and price are server-controlled.

Impact: Protected order fields or account attributes may be changed.

Remediate: Bind an explicit DTO/allowlist and set sensitive fields only in trusted server logic.

VULN_PATH_TRAVERSAL_DOWNLOAD — enabled

Implemented

Definition: A requested filename is joined to an upload directory without a safe containment check.

Safe test: In a disposable filesystem, unit-test that path resolution rejects any file outside a dedicated temporary upload root; do not read real host files.

Impact: Files accessible to the web process may be disclosed.

Remediate: Use opaque file IDs, canonicalize paths, and verify resolved paths remain inside the permitted directory.

VULN_SSRF — enabled

Implemented; admin path

Definition: A server fetches a user-provided URL without destination controls.

Safe test: In an isolated harness, point only to a local mock server and assert the application refuses unapproved destinations; never target cloud metadata or public hosts.

Impact: The server could be induced to access internal resources or make unintended requests.

Remediate: Allowlist destinations, block private/link-local ranges after DNS resolution, restrict protocols, and limit redirects/timeouts.

VULN_INSECURE_DESERIALIZATION — enabled

Misnamed / unused toggle

Definition: Deserialization of untrusted data can instantiate objects or trigger unsafe behavior.

Safe test: The unsafe remember-cookie path is governed by `VULN_WEAK_REMEMBER_ME`; verify source only and do not craft serialized objects.

Impact: Depending on available classes, object injection can cause unauthorized state changes or file operations.

Remediate: Do not deserialize untrusted data; use signed opaque tokens or JSON with strict schema validation.

VULN_WEAK_REMEMBER_ME — enabled

Implemented

Definition: A remember cookie is decoded and unserialized; supplied fields may populate session identity/role.

Safe test: In a local test, verify invalid cookies are rejected and no deserializer is used; don't construct objects or attempt identity changes.

Impact: Forged cookie state may impersonate users or elevate privileges; the source also defines a file-writing destructor class.

Remediate: Remove object deserialization; use random server-side tokens stored hashed and rotate/revoke them.

VULN_PREDICTABLE_RESET_TOKEN — enabled

Not implemented

Definition: Password-reset tokens are guessable or generated from predictable data.

Safe test: Verify no reset-token workflow exists; if added, unit-test cryptographically random token generation and expiry.

Impact: An account could be reset without its owner's authorization.

Remediate: Generate high-entropy random tokens, store hashes, expire quickly, and invalidate after use.

VULN_RACE_CONDITION — enabled

Not implemented

Definition: Concurrent requests can interleave non-atomic state checks and updates.

Safe test: Review transaction boundaries and constraints; no race exercise is configured in the PHP app.

Impact: A one-time action may be applied more than once.

Remediate: Use atomic updates, transactions, row locks, and uniqueness/idempotency constraints.

VULN_RACE_MULTI_ENDPOINT — enabled

Not implemented

Definition: Different endpoints update shared state without coordinating atomicity.

Safe test: Inspect all writers to the same record and add deterministic concurrency tests only in a disposable database.

Impact: Parallel flows may bypass limits or overwrite valid state.

Remediate: Put invariants in database transactions and constraints shared across endpoints.

VULN_RACE_SINGLE_PACKET — enabled

Not implemented

Definition: A timing-sensitive race is triggered by near-simultaneous requests.

Safe test: No current challenge is present; test synchronization invariants in unit/integration tests, not against a live service.

Impact: A concurrent operation may violate a one-time business rule.

Remediate: Make the state transition atomic and idempotent.

VULN_RACE_DOUBLE_SPEND — enabled

Not implemented

Definition: A balance or credit is consumed more than once because checks and deductions race.

Safe test: Review balance updates for an atomic conditional update; no spend-race lab route exists.

Impact: Account balances or inventory may become inconsistent.

Remediate: Use a transactional ledger and atomic compare-and-update constraints.

VULN_RACE_VOUCHER_OVERRUN — enabled

Not implemented

Definition: Concurrent voucher redemptions exceed a usage limit.

Safe test: No voucher workflow is implemented; if introduced, test limits with a local concurrency harness and synthetic vouchers.

Impact: Discounts may exceed the intended redemption count.

Remediate: Enforce redemption limits atomically in the database.

VULN_WORKFLOW_STEP_SKIP — enabled

Not implemented

Definition: A later workflow action is accepted without required earlier approvals or checks.

Safe test: Map allowed state transitions and assert invalid transitions are rejected; no matching challenge exists.

Impact: Orders or approvals may reach an unauthorized state.

Remediate: Validate state transitions server-side and audit who performed each step.

VULN_ROUNDING_ERROR — enabled

Not implemented

Definition: Floating-point or inconsistent rounding produces incorrect monetary totals.

Safe test: Add unit tests for documented currency rounding rules; no rounding challenge was found.

Impact: Repeated calculations may drift, creating over/under-charges.

Remediate: Use integer minor units or decimal arithmetic and round at defined boundaries.

VULN_INTEGER_OVERFLOW — enabled

Legacy quantity path

Definition: The legacy quantity update applies no upper bound to the integer quantity.

Safe test: In an isolated test, assert quantities above the documented maximum are rejected and no order/cart data outside the test account changes.

Impact: Oversized quantities may cause resource or business-logic errors; main cart flow is separate.

Remediate: Validate integer range, stock, and type at the server boundary.

VULN_NEGATIVE_REFUND — enabled

Not implemented

Definition: Refund logic accepts a negative or otherwise invalid amount.

Safe test: Confirm no refund handler exists; if added, test boundary validation with a fake ledger.

Impact: Invalid accounting entries could increase balances or corrupt the ledger.

Remediate: Validate positive bounded amounts and reconcile against original settled transactions.

VULN_LOYALTY_ABUSE — enabled

Not implemented

Definition: Loyalty points or benefits can be earned or redeemed outside policy.

Safe test: Confirm there is no points ledger or redemption route in the current app.

Impact: Benefits may be improperly issued or transferred.

Remediate: Use an auditable ledger with server-validated eligibility and idempotent redemption.

VULN_HORIZONTAL_PRIV_ESC — enabled

Not implemented as named

Definition: A user can access another user's resources at the same privilege level.

Safe test: Use paired synthetic accounts and assert ownership checks; order-view IDOR is documented separately.

Impact: Another user's private information or actions may be exposed.

Remediate: Enforce object-level authorization on every request.

VULN_VERTICAL_PRIV_ESC — enabled

Not implemented as named

Definition: A lower-privilege account can perform an administrative action.

Safe test: Check role enforcement with synthetic learner/admin accounts; do not use a real privileged account.

Impact: Unauthorized administrative actions may be possible.

Remediate: Deny by default and authorize each action server-side.

VULN_HTTP_METHOD_SWAP — enabled

Not implemented

Definition: Changing an HTTP method bypasses a control or changes semantics unexpectedly.

Safe test: Review route method restrictions; no method-swap challenge was established.

Impact: A protected action may be invoked through an unintended request method.

Remediate: Enforce methods at the route and authorization layers; make state-changing operations explicit.

VULN_URL_QUIRKS — enabled

Implemented; server-dependent

Definition: An admin check behaves differently for a path variant, depending on request-path handling.

Safe test: On localhost only, compare authorization outcomes for canonical versus normalized route paths using a synthetic learner; verify server routing behavior.

Impact: A path interpretation mismatch may skip an access check.

Remediate: Use framework/router route identity, canonicalize paths once, and authorize independently of URL spelling.

VULN_SPOOFABLE_ACCESS_CONTROL — enabled

Not implemented

Definition: Authorization relies on a client-controlled identity or role value.

Safe test: Inspect access decisions and confirm they derive from server-side session/user records.

Impact: A client may impersonate another identity or role.

Remediate: Treat client claims as untrusted; validate signed identity and roles on the server.

VULN_HTTP_VERB_TAMPERING — enabled

Dead helper check

Definition: An access check is bypassed based on an HTTP method variation.

Safe test: The check is inside an access helper no current page calls; source review confirms it is not a reachable exercise.

Impact: If wired incorrectly, method choice could skip authorization.

Remediate: Use route-level method constraints and a common authorization guard.

Browser, client-side, and CSRF

The PHP page source does not establish the advertised XSS or JavaScript exercises. CSRF weaknesses exist on selected actions independently of the named toggle.

VULN_CSRF_METHOD_OVERRIDE — enabled

Not implemented

Definition: An alternate method indicator changes a request method and bypasses protection.

Safe test: Check for method-override middleware and verify CSRF protection is applied after method normalization; none is configured here.

Impact: State-changing actions could be reached in an unintended way.

Remediate: Avoid unnecessary method overrides and apply CSRF checks to the effective method.

VULN_CSRF_TOKEN_LEAK — enabled

Not implemented

Definition: Anti-CSRF tokens are exposed to an untrusted origin or included in URLs/logs.

Safe test: Inspect forms, links, referrers, and logs for token exposure; no token-leak exercise was found.

Impact: A leaked token may enable forged state changes.

Remediate: Keep tokens out of URLs, scope them to sessions, and apply restrictive referrer policy.

VULN_CSRF_FAIL_OPEN — enabled

Not implemented as toggle

Definition: A missing or invalid anti-CSRF token is accepted.

Safe test: Verify each state-changing handler rejects missing/invalid tokens; current selected actions lack protection.

Impact: Browser-held credentials may be used for unwanted actions.

Remediate: Fail closed on token validation and test all state-changing routes.

VULN_CLICKJACKING_DRAG — enabled

Not implemented

Definition: A page can be framed and visually disguised to trick user interaction.

Safe test: Inspect response headers for frame restrictions; no drag-based browser challenge exists.

Impact: A user may click a hidden or disguised control.

Remediate: Use CSP `frame-ancestors` and avoid sensitive UI in untrusted frames.

VULN_DOM_CLOBBERING — enabled

Not implemented

Definition: Attacker-controlled markup changes DOM names/properties used by client code.

Safe test: Review HTML and JavaScript data flow; no current DOM-clobbering exercise is present.

Impact: Client logic may use an unintended element or value.

Remediate: Avoid named-property reliance, validate DOM references, and sanitize markup.

VULN_CLIENT_PROTOTYPE_POLLUTION — enabled

Not implemented

Definition: Untrusted object keys alter shared JavaScript prototypes.

Safe test: Search client code for unsafe recursive merges; no application JavaScript exercise is mapped to this toggle.

Impact: Client behavior or security checks may be altered globally.

Remediate: Use safe merge routines, reject prototype-related keys, and use plain data objects.

VULN_POSTMESSAGE_NO_ORIGIN — enabled

Not implemented

Definition: A message handler accepts cross-window messages without validating sender origin.

Safe test: Search for `message` listeners and verify strict origin checks; none is established in the PHP pages.

Impact: An untrusted site may influence the receiving page's actions.

Remediate: Validate origin and message schema, and restrict target origins when sending.

VULN_UNSAFE_EVAL — enabled

Not implemented

Definition: Untrusted input is evaluated as JavaScript code.

Safe test: Search scripts for dynamic evaluation; no matching client-side exercise was found.

Impact: Input may execute with the page's browser privileges.

Remediate: Avoid dynamic evaluation and use data parsing plus strict schemas.

VULN_CLIENT_SIDE_VAL_TRUST — enabled

Not implemented

Definition: A client-side validation result is trusted as authoritative by the server.

Safe test: Inspect server handlers to ensure all business rules are repeated server-side; no challenge is mapped.

Impact: Invalid client state may be accepted by the server.

Remediate: Treat browser validation as UX only; enforce rules on the server.

Uploads, network access, and information exposure

VULN_UNRESTRICTED_FILE_UPLOAD — enabled

Not present; uploads are validated

Definition: Arbitrary file content is accepted and stored where it can be executed or served unsafely.

Safe test: Review the upload helper and test only synthetic allowed image files plus rejected invalid fixtures. Current helper checks image type and renames files.

Impact: Unrestricted uploads can enable stored content attacks or server-side execution.

Remediate: Keep content validation, random server filenames, size limits, and non-executable storage.

VULN_EXPOSED_SOURCE_MAP — enabled

Not implemented

Definition: Public source maps reveal original source or embedded development metadata.

Safe test: Review build artifacts for source maps and ensure none contains secrets; no source-map challenge is mapped.

Impact: Internal code and accidental source-embedded values may be disclosed.

Remediate: Exclude maps from public production artifacts or protect them; never embed secrets in client code.

VULN_COOKIE_DEBUG_MODE — enabled

Not implemented

Definition: Debug behavior exposes cookie or session internals.

Safe test: Search responses/logging for cookie values; no debug-cookie handler was found.

Impact: Session material may be disclosed.

Remediate: Never log secrets; redact cookies and disable debug output in deployment.

VULN_AUTOCOMPLETE_ENABLED — enabled

Not implemented as a vulnerability

Definition: Sensitive forms allow browser autofill where policy may require reduced retention.

Safe test: Inspect form autocomplete attributes; this is not a verified server-side challenge.

Impact: Shared-device users may see previously entered values.

Remediate: Apply appropriate autocomplete semantics and avoid collecting unnecessary sensitive data.

VULN_MISSING_CACHE_CONTROL — enabled

Not demonstrated

Definition: Sensitive responses may be stored by shared or browser caches.

Safe test: Inspect headers on authenticated pages; no toggle-controlled cache behavior was found.

Impact: Private content may remain available in a shared cache or browser history.

Remediate: Set suitable `Cache-Control: no-store` on sensitive responses.

VULN_VERBOSE_API_JSON — enabled

Not demonstrated

Definition: API responses include internal fields or diagnostic data not needed by callers.

Safe test: Review response schemas and assert only public fields are returned; no verbose JSON challenge is established.

Impact: Internal identifiers or personal data may be exposed.

Remediate: Use explicit response DTOs and minimize returned fields.

VULN_VERBOSE_ERRORS — disabled

Disabled

Definition: Detailed runtime errors are shown to users.

Safe test: Confirm the switch is false and error display is disabled; the current app logs errors server-side.

Impact: Stack traces may expose paths, queries, or secrets.

Remediate: Return generic errors and retain sanitized server-side diagnostics.

VULN_EXPOSED_BACKUPS — enabled

Backup file present; access server-dependent

Definition: Backup/configuration files are present in a web-served directory.

Safe test: Inspect the local build artifact list; do not request sensitive files from a deployed host.

Impact: Backups may disclose source or configuration values.

Remediate: Exclude backups from image/build artifacts and keep secrets in environment configuration.

VULN_HARDCODED_CREDS — enabled

Not demonstrated as a toggle

Definition: Reusable credentials are embedded in source or deployment artifacts.

Safe test: Review synthetic demo seed values and deployment files; do not test credentials against external services.

Impact: Disclosed credentials may grant access to accounts or services.

Remediate: Use secret management, remove production credentials from source, and rotate exposed secrets.

VULN_DEBUG_ENDPOINTS — enabled

Implemented

Definition: A diagnostic route exposes PHP configuration information.

Safe test: In local source/tests, verify the debug route is inaccessible in a production-mode build; never expose its output publicly.

Impact: Runtime details and configuration may aid further attacks.

Remediate: Remove or protect diagnostics and ensure deployment access gate remains enabled.

VULN_MISSING_HEADERS — enabled

Implemented

Definition: Common browser security headers are omitted by the shared page header.

Safe test: Inspect response headers on localhost and assert CSP/frame, nosniff, and other selected policies are present in a hardened build.

Impact: Browser protections against framing, content sniffing, and script injection are reduced.

Remediate: Set a reviewed header policy centrally and test actual responses.

Money, input, normalization, and remaining switches

VULN_UNICODE_NORMALIZATION — enabled

Not implemented

Definition: Unicode-equivalent strings are treated inconsistently by validation and lookup.

Safe test: Add unit tests for canonical Unicode normalization at identifier boundaries; no challenge exists.

Impact: Lookalike or equivalent names may evade duplicate or access checks.

Remediate: Normalize identifiers consistently before validation, storage, and comparison.

VULN_NULL_BYTE_INJECTION — enabled

Not implemented

Definition: A null byte truncates or changes interpretation across components.

Safe test: Verify inputs with null bytes are rejected by validation tests; no file challenge is mapped.

Impact: Validation and downstream file handling may disagree.

Remediate: Reject control characters and use safe filesystem APIs with canonical paths.

VULN_CASE_SENSITIVITY_BYPASS — enabled

Not implemented; search is case-insensitive

Definition: Case handling differs between a security check and the protected lookup.

Safe test: Compare case-insensitive lookups and authorization normalization; product search is intentionally case-insensitive.

Impact: A case variation could evade a filter or uniqueness check.

Remediate: Canonicalize consistently and apply checks to canonical values.

VULN_TRAILING_DOT_SLASH — enabled

Not implemented

Definition: Trailing path syntax is interpreted inconsistently by routers or filesystems.

Safe test: Review route canonicalization and test a local router's normalized path representation; no exercise is present.

Impact: Path-based restrictions may be bypassed if components disagree.

Remediate: Canonicalize once and authorize using the normalized route/resource.

VULN_SQL_COMMENT_FILTER_BYPASS — enabled

Not implemented

Definition: A SQL filter can be bypassed by alternative comment syntax.

Safe test: Confirm no SQL comment filter is used; replace any such filter with bound parameters.

Impact: If a filter is the only defense, query structure may be altered.

Remediate: Never filter SQL syntax as a substitute for prepared statements.

Coverage and current limitations

The switch names are not a promise that the challenge is implemented. The current inventory identifies 14 enabled toggles connected to relevant code behavior; several additional weaknesses exist independently of their similarly named toggle. Most of the remaining enabled switches have no working route/handler challenge in this PHP build. Six switches are disabled. For the authoritative mapping and reachability caveats, see `VULNERABILITIES.md`.

Safe Lab simulation certificates can be claimed individually for each of the 69 configured toggles after its in-memory check passes. They certify simulation completion only. Separately, only login SQL injection has automatic exploit detection and an identity-claim flow; flag submissions can create awards but do not prove the matching vulnerability was exploited. Learner name/email are self-reported, and email is not verified. Valid claims produce a PDF certificate; duplicate claims for the same challenge/email are blocked across demo accounts.

General remediation sequence: reproduce only in a local disposable environment; add a regression test;

implement server-side validation, authorization, parameterization, or safe API use; run PHP lint and relevant tests; inspect the change; keep deployment protected; and document any behavior change.

Sources: current PHP configuration and handlers, and `VULNERABILITIES.md`. Reference scope is the PHP application only, not the separate React or Django projects in the repository.